AI Applications Lecture 11

Introduction to visual data generation AI

SUZUKI, Atsushi[a]

[a] I thank my collaborator, Jing WANG, for cooperating in creating the teaching materials.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. In which sense are visual data generation different from Chat AI?

4. Why do we avoid constructing a probability mass function?

5. How does a practical visual data generation pipeline avoid the issues?

6. Summary and Future Plans

Chat AI's pipeline isn't everything.

In the first half of this course, we focused on **Natural Language Generation**, covering concepts such as the **pipeline**, **tokenization**, **sampling**, **special tokens**, and **chat templates**.

So,

Chat AI's pipeline isn't everything.

In the first half of this course, we focused on **Natural Language Generation**, covering concepts such as the **pipeline**, **tokenization**, **sampling**, **special tokens**, and **chat templates**.

So, do we use the same pipeline in visual data (e.g., image, video, 3D rendering) generation, etc.? -

Chat AI's pipeline isn't everything.

In the first half of this course, we focused on **Natural Language Generation**, covering concepts such as the **pipeline**, **tokenization**, **sampling**, **special tokens**, and **chat templates**.

So, do we use the same pipeline in visual data (e.g., image, video, 3D rendering) generation, etc.? - No.

- Why not?
- What kind of a pipeline do we use?

Learning outcomes

At the end of this lecture, you will be able to

Learning outcomes

At the end of this lecture, you will be able to

- Explain the mathematical characteristics of the **visual data generation** task and how it differs from natural language generation.

Learning outcomes

At the end of this lecture, you will be able to

- Explain the mathematical characteristics of the **visual data generation** task and how it differs from natural language generation.
- Explain what kind of **pipeline** constitutes practical **text-to-visual-data AI** and why.

Learning outcomes

At the end of this lecture, you will be able to

- Explain the mathematical characteristics of the **visual data generation** task and how it differs from natural language generation.
- Explain what kind of **pipeline** constitutes practical **text-to-visual-data AI** and why.
- Have a general **pipeline design** principle so you can handle unknown tasks in the future.

Revisiting the grand principle

The goal of AI is to learn the **appropriate relationship**, i.e., a **function**, between input and output.

Revisiting the grand principle

The goal of AI is to learn the **appropriate relationship**, i.e., a **function**, between input and output.

This is the same for both natural language generation and image generation.

Outline

1. **Introduction**
2. Preliminaries: Mathematical Notations
3. In which sense are visual data generation different from Chat AI?
4. Why do we avoid constructing a probability mass function?
5. How does a practical visual data generation pipeline avoid the issues?
6. Summary and Future Plans

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. In which sense are visual data generation different from Chat AI?

4. Why do we avoid constructing a probability mass function?

5. How does a practical visual data generation pipeline avoid the issues?

6. Summary and Future Plans

Preliminaries: mathematical notations

Set:

- Sets: $\mathcal{A}$
- Membership: $x \in \mathcal{A}$
- Empty set: $\{\}$
- Roster notation: $\{a, b, c\}$
- Set-builder: $\{x \in \mathcal{A} | P(x)\}$
- Cardinality: $|\mathcal{A}|$
- Real numbers: $\mathbb{R}, \mathbb{R}_{>0}, \mathbb{R}_{\geq 0}$
- Integers: $\mathbb{Z}, \mathbb{Z}_{>0}, \mathbb{Z}_{\geq 0}$
- Integer range: $[1, k]_{\mathbb{Z}} \coloneqq \{1, \ldots, k\}$

Function:

- $f : \mathcal{X} \to \mathcal{Y}$: f maps from $\mathcal{X}$ to $\mathcal{Y}$.
- $y = f(x)$: The output of f for input x.

Definition:

- $(\mathrm{LHS}) \coloneqq (\mathrm{RHS})$: Left side is defined by the right side.

Preliminaries: mathematical notations

Sequence: Denoted by $\boldsymbol{a} = (a_1, a_2, \dots)$.

- A function $\boldsymbol{a} : [1, n]_{\mathbb{Z}} \to \mathcal{A}$.
- Length is denoted by $|\boldsymbol{a}|$.

Vector: Denoted by $\boldsymbol{v}$.

- A column of numbers, $\boldsymbol{v} \in \mathbb{R}^n$.
- i-th element is v_i.

Matrix: Denoted by $\boldsymbol{A}$.

- $m \times n$ matrix: $\boldsymbol{A} \in \mathbb{R}^{m,n}$.
- (i, j)-th element is $a_{i,j}$.
- Transpose: $\boldsymbol{A}^\top$.

Tensor: Denoted by $\underline{\boldsymbol{A}}$.

- Simply a multi-dimensional array.
- Vector $\to$ 1st-order, Matrix $\to$ 2nd-order.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. In which sense are visual data generation different from Chat AI?

4. Why do we avoid constructing a probability mass function?

5. How does a practical visual data generation pipeline avoid the issues?

6. Summary and Future Plans

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. In which sense are visual data generation different from Chat AI?

4. Why do we avoid constructing a probability mass function?

5. How does a practical visual data generation pipeline avoid the issues?

6. Summary and Future Plans

Question: In which sense, visual data generation is different from text generation?

Characteristics of image generation as a task

Recall that AI aims to achieve an appropriate input-output relation.

Text input: $\boldsymbol{y} = (y_1, y_2, ..., y_m) \in \mathcal{V}^*$, where $\mathcal{V}$ is the token set (vocabulary).

Text output: $\boldsymbol{y} = (y_1, y_2, ..., y_n) \in \mathcal{V}^*$. The length n **depends on the input**.

Characteristics of image generation as a task

Recall that AI aims to achieve an appropriate input-output relation.

Text input: $\boldsymbol{y} = (y_1, y_2, ..., y_m) \in \mathcal{V}^*$, where $\mathcal{V}$ is the token set (vocabulary).

Text output: $\boldsymbol{y} = (y_1, y_2, ..., y_n) \in \mathcal{V}^*$. The length n **depends on the input**.

Let's take the image generation as a visual data generation example:

Characteristics of image generation as a task

Recall that AI aims to achieve an appropriate input-output relation.

Text input: $\boldsymbol{y} = (y_1, y_2, ..., y_m) \in \mathcal{V}^*$, where $\mathcal{V}$ is the token set (vocabulary).

Text output: $\boldsymbol{y} = (y_1, y_2, ..., y_n) \in \mathcal{V}^*$. The length n **depends on the input**.

Let's take the image generation as a visual data generation example:

Image output: $\underline{\boldsymbol{Y}} \in \{0, 1, ..., 255\}^{H,W,C}$, where H and W are the height and width of the output image and C is the number of channels (e.g., red green, blue). The output shape

Characteristics of image generation as a task

Recall that AI aims to achieve an appropriate input-output relation.

Text input: $\boldsymbol{y} = (y_1, y_2, ..., y_m) \in \mathcal{V}^*$, where $\mathcal{V}$ is the token set (vocabulary).

Text output: $\boldsymbol{y} = (y_1, y_2, ..., y_n) \in \mathcal{V}^*$. The length n **depends on the input**.

Let's take the image generation as a visual data generation example:

Image output: $\underline{\boldsymbol{Y}} \in \{0, 1, ..., 255\}^{H,W,C}$, where H and W are the height and width of the output image and C is the number of channels (e.g., red green, blue). The output shape does **NOT depend on the input prompt**.

Aspects where image generation is easier

Aspects where image generation is easier

- **Pre-determination of Output Dimensions**: In image output applications, the output shape (H, W, C) is often explicitly given. So, it is **often unnecessary to dynamically determine the output length**.

Aspects where image generation is easier

- **Pre-determination of Output Dimensions**: In image output applications, the output shape (H, W, C) is often explicitly given. So, it is **often unnecessary to dynamically determine the output length**.
- **Tolerance to Local Errors**: Pixel-level errors are often acceptable. There are usually no strong **discrete constraints** on the adjacency of pixels.
  - In contrast...,

Aspects where image generation is easier

- **Pre-determination of Output Dimensions**: In image output applications, the output shape (H, W, C) is often explicitly given. So, it is **often unnecessary to dynamically determine the output length**.
- **Tolerance to Local Errors**: Pixel-level errors are often acceptable. There are usually no strong **discrete constraints** on the adjacency of pixels.
  - In contrast..., in natural language, strong **local constraints** exist in token sequences due to grammar, etc.
  - Example:

Aspects where image generation is easier

- **Pre-determination of Output Dimensions**: In image output applications, the output shape (H, W, C) is often explicitly given. So, it is **often unnecessary to dynamically determine the output length**.
- **Tolerance to Local Errors**: Pixel-level errors are often acceptable. There are usually no strong **discrete constraints** on the adjacency of pixels.
  - In contrast..., in natural language, strong **local constraints** exist in token sequences due to grammar, etc.
  - Example: Although "a" and "the" are individually frequent, "the" almost never appears immediately after "a".

Aspects where image generation is harder

- **Enormous Output Dimensionality**: Even on a small scale, if $H = W = 256,\ C = 3$, the dimensionality is $196,608$. In contrast, many natural language applications can be covered with an output of $10,000$ tokens.

Aspects where image generation is harder

- **Enormous Output Dimensionality**: Even on a small scale, if $H = W = 256,\ C = 3$, the dimensionality is $196,608$. In contrast, many natural language applications can be covered with an output of $10,000$ tokens.
- **Need for Information Supplementation**: Particularly in text-to-image, the input text prompt may be a few dozen to a hundred tokens. The input information is insufficient to determine the output image.
  - Example: Suppose the input is “Generate an image of dog.” To generate, we need to supplement the information of the dog’s
    - color,
    - breed,
    - size, etc.

Difference between visual data generation and text generation

Question: In which sense, visual data generation is different from text generation?

Answer:

Difference between visual data generation and text generation

Question: In which sense, visual data generation is different from text generation?

Answer:

- Visual data can be extremely high-dimensional.
- The input's information may be insufficient to determine an output.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. In which sense are visual data generation different from Chat AI?

4. Why do we avoid constructing a probability mass function?

5. How does a practical visual data generation pipeline avoid the issues?

6. Summary and Future Plans

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. In which sense are visual data generation different from Chat AI?

4. Why do we avoid constructing a probability mass function?

5. How does a practical visual data generation pipeline avoid the issues?

6. Summary and Future Plans

Question: In text generation, we constructed a conditional probability mass function (PMF) and sampled a token iteratively. Why

Question: In text generation, we constructed a conditional probability mass function (PMF) and sampled a token iteratively. Why do we avoid conditional PMF methods?

Review: text generation by constructing conditional PMF

In natural language generation, a probabilistic language model (conditional probability mass function) over the vocabulary $\mathcal{V}$

$$P(y_t \mid \boldsymbol{y}_{<t}, \boldsymbol{x}), \quad t = 1, 2, \ldots \tag{1}$$

is defined by a neural network, and a sequence is generated by **sequential sampling**. We can stop output when the **EOS (end-of-sequence)** is sampled.

Review: text generation by constructing conditional PMF

In natural language generation, a probabilistic language model (conditional probability mass function) over the vocabulary $\mathcal{V}$

$$P(y_t \mid \boldsymbol{y}_{<t}, \boldsymbol{x}), \quad t = 1, 2, \ldots \tag{1}$$

is defined by a neural network, and a sequence is generated by **sequential sampling**. We can stop output when the **EOS (end-of-sequence)** is sampled.

Pros 1: **The output length can depend on the input**.

Pros 2: **Local constraints** can be easily realized.

What is the NN output dimensionality?

Recall the softmax approach:

- Get the score vector from NN: $\boldsymbol{s}^{(t)} \leftarrow f_{\boldsymbol{\theta}}(\boldsymbol{x} \cdot \boldsymbol{y}_{<t})$
- Get each token's probability mass $p_v^{(t)} \leftarrow \frac{\exp\left(s_v^{(t)}\right)}{\sum_{u \in \mathcal{U}} \exp\left(s_u^{(t)}\right)}$.

Here,

What is the NN output dimensionality?

Recall the softmax approach:

- Get the score vector from NN: $\boldsymbol{s}^{(t)} \leftarrow f_{\boldsymbol{\theta}}(\boldsymbol{x} \cdot \boldsymbol{y}_{<t})$
- Get each token's probability mass $p_v^{(t)} \leftarrow \frac{\exp\left(s_v^{(t)}\right)}{\sum_{u \in \mathcal{U}} \exp\left(s_u^{(t)}\right)}$.

Here, the dimension of the NN output $\boldsymbol{s}^{(t)}$ is the same as the vocabulary, i.e., the number of possible outputs.

Constructing the output PMF is impossible in image generation

In the softmax-PMF scheme, the NN needs to output a vector whose dimension is the same as the number of possible outputs. So,

Constructing the output PMF is impossible in image generation

In the softmax-PMF scheme, the NN needs to output a vector whose dimension is the same as the number of possible outputs. So, what if we apply it in image generation?

If $H = W = 256$, $C = 3$, then

Constructing the output PMF is impossible in image generation

In the softmax-PMF scheme, the NN needs to output a vector whose dimension is the same as the number of possible outputs. So, what if we apply it in image generation?

If $H = W = 256$, $C = 3$, then the image has $HMC = 196,608$ pixels, i.e., is $196,608$-dimensional. Each element

Constructing the output PMF is impossible in image generation

In the softmax-PMF scheme, the NN needs to output a vector whose dimension is the same as the number of possible outputs. So, what if we apply it in image generation?

If $H = W = 256$, $C = 3$, then the image has $HMC = 196,608$ pixels, i.e., is $196,608$-dimensional. Each element may take a value in $\{0, 1, ..., 255\}$. The number of possible values is:

Constructing the output PMF is impossible in image generation

In the softmax-PMF scheme, the NN needs to output a vector whose dimension is the same as the number of possible outputs. So, what if we apply it in image generation?

If $H = W = 256$, $C = 3$, then the image has $HMC = 196,608$ pixels, i.e., is $196,608$-dimensional. Each element may take a value in $\{0, 1, ..., 255\}$. The number of possible values is:

$$50,331,648$$

Constructing the output PMF is impossible in image generation

In the softmax-PMF scheme, the NN needs to output a vector whose dimension is the same as the number of possible outputs. So, what if we apply it in image generation?

If $H = W = 256$, $C = 3$, then the image has $HMC = 196,608$ pixels, i.e., is $196,608$-dimensional. Each element may take a value in $\{0, 1, ..., 255\}$. The number of possible values is:

$$50,331,648$$

Implementing a 50,331,648-dimensional-output NN is NOT practical.

What if we construct conditional PMF?

We may serialize all the pixels and define a conditional PMF, as we did in text generation. However,

What if we construct conditional PMF?

We may serialize all the pixels and define a conditional PMF, as we did in text generation. However,

- It requires HWC-times sampling (e.g., $HWC = 196,608$). Not practical

What if we construct conditional PMF?

We may serialize all the pixels and define a conditional PMF, as we did in text generation. However,

- It requires HWC-times sampling (e.g., $HWC = 196,608$). Not practical
- It destroy the 2D-3ch structure of a color image.

Benefits of the conditional PMF method is not leveraged

Image generation's local constraints are not strong as text generation, the advantages of constructing explicit conditional PMF are not well leveraged.

Why do we avoid constructing a PMF in visual data generation? - Answer

Question: Why

Why do we avoid constructing a PMF in visual data generation? - Answer

Question: Why do we avoid conditional PMF methods in visual data generation?

Answer: Conditional PMF methods requires a NN whose output dimension is the same as the number of possible outputs. The number can be extremely large in visual data generation.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. In which sense are visual data generation different from Chat AI?

4. Why do we avoid constructing a probability mass function?

5. How does a practical visual data generation pipeline avoid the issues?

6. Summary and Future Plans

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. In which sense are visual data generation different from Chat AI?

4. Why do we avoid constructing a probability mass function?

5. How does a practical visual data generation pipeline avoid the issues?

6. Summary and Future Plans

Question: Visual data generation has a high-dimensional output issue. Constructing a PMF does not work. How

Question: Visual data generation has a high-dimensional output issue. Constructing a PMF does not work. How does a practical visual data generation pipeline avoid the issues?

What should we achieve?

What should we achieve?

- We need to handle high-dimensional output.

What should we achieve?

- We need to handle high-dimensional output.
- To compensate for the insufficient information in an input prompt, we need an output diversity.

Hence, we want the AI to output a **high-dimensional** vector **stochastically**.

Problem formulation

Let's consider text-to-image as a representative application.

Input: a token sequence $\boldsymbol{x} \in \mathcal{V}^*$, where $\mathcal{V}$ is the vocabulary.

Output: Image tensor $\underline{\boldsymbol{Y}} \in \{0, 1, ..., 255\}^{H,W,C}$.

Text-to-image pipeline: A parametric function Text2Img$_{\boldsymbol{\theta}}$ that receives an input text a sequence of random variables and returns an image tensor.

The output image $\underline{\boldsymbol{Y}}$ is given by

$$\underline{\boldsymbol{Y}} \leftarrow \text{Text2Img}_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{u}),$$

where $\boldsymbol{u} = u_1, u_2, ..., u_k$ are random variables. The number k of random variables consumed by Text2Img$_{\boldsymbol{\theta}}$ may be fixed but can depend on the input $\boldsymbol{x}$.

How can we realize stochastic high-dimensional output?

Constructing a PMF is not good idea in high-dimensional output.

Instead of PMF-based methods, we construct a stochastic output by

How can we realize stochastic high-dimensional output?

Constructing a PMF is not good idea in high-dimensional output.

Instead of PMF-based methods, we construct a stochastic output by **pushforwarding a base measure**.

How can we realize stochastic high-dimensional output?

Constructing a PMF is not good idea in high-dimensional output.

Instead of PMF-based methods, we construct a stochastic output by **pushforwarding a base measure**.

What's the difference between constructing a PMF and pushforward a base measure? How should we choose a better one, depending on the situation?

How can we realize stochastic high-dimensional output?

Constructing a PMF is not good idea in high-dimensional output.

Instead of PMF-based methods, we construct a stochastic output by **pushforwarding a base measure**.

What's the difference between constructing a PMF and pushforward a base measure? How should we choose a better one, depending on the situation?

Let's calculate some examples.

How can we realize stochastic high-dimensional output?

Constructing a PMF is not good idea in high-dimensional output.

Instead of PMF-based methods, we construct a stochastic output by **pushforwarding a base measure**.

What's the difference between constructing a PMF and pushforward a base measure? How should we choose a better one, depending on the situation?

Let's calculate some examples.

One technical difference is

- The PMF method uses the source random variable in the output phase, while

How can we realize stochastic high-dimensional output?

Constructing a PMF is not good idea in high-dimensional output.

Instead of PMF-based methods, we construct a stochastic output by **pushforwarding a base measure**.

What's the difference between constructing a PMF and pushforward a base measure? How should we choose a better one, depending on the situation?

Let's calculate some examples.

One technical difference is

- The PMF method uses the source random variable in the output phase, while The pushforward method uses the source random variable in the in phase.

Constructing a random variable by a PMF

Constructing a random variable by a PMF

Example (Constructing a random variable by a PMF)

Suppose that U is a uniform random variable taking a value on $[0, 1)$. Let $P : \{0, 1, 2, 3\} \to [0, 1]$ be a PMF, defined by $P(0) = 0.5, P(1) = 0.1, P(2) = 0.3, P(3) = 0.1$.

Consider the following algorithm:

- Compute the cumulative distribution function $F(i) = \sum_{i'=0}^{i} P(i')$.
- Sample $U \sim \mathsf{Uniform}([0, 1))$.
- Output $Y := \min\{i \in \{0, 1, 2, 3\} \mid U < F(i)\}$.

What is the distribution of Y?

Constructing a random variable by a PMF

Answer:

$$Y = \begin{cases} 0 & \text{if } U \in [0, 0.5), \\ 1 & \text{if } U \in [0.5, 0.6), \\ 2 & \text{if } U \in [0.6, 0.9), \\ 3 & \text{if } U \in [0.9, 1.0). \end{cases}$$

Hence, $\text{Probability}(Y = i) = P(i)$. That is, Y follows the distribution defined by the PMF P.

Constructing a random variable by pushforwarding a measure - continuous

Constructing a random variable by pushforwarding a measure - continuous

Example (Pushforward measure of a continuous function)

Suppose that U is a uniform random variable taking a value on $[0, 1)$.

Define $f : [0, 1) \to \mathbb{R}$ by the following:

$$f(u) = \begin{cases} \frac{1}{2}u & \text{if } u \in [0, \frac{1}{4}), \\ 3(u - \frac{1}{4}) + \frac{1}{8} & \text{if } u \in [\frac{1}{4}, \frac{3}{4}), \\ \frac{1}{2}u + \frac{5}{4} & \text{if } u \in [\frac{3}{4}, 1). \end{cases}$$

Construct the output Y by $Y = f(U)$. What is the distribution of Y?

Constructing a random variable by pushforwarding a measure - continuous

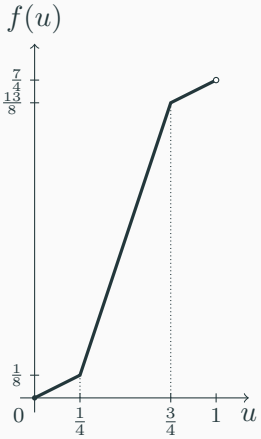
Example (Pushforward measure of a continuous function)

Suppose that U is a uniform random variable taking a value on $[0, 1)$.

Define $f : [0, 1) \to \mathbb{R}$ by the following:

$$f(u) = \begin{cases} \frac{1}{2}u & \text{if } u \in [0, \frac{1}{4}), \\ 3(u - \frac{1}{4}) + \frac{1}{8} & \text{if } u \in [\frac{1}{4}, \frac{3}{4}), \\ \frac{1}{2}u + \frac{5}{4} & \text{if } u \in [\frac{3}{4}, 1). \end{cases}$$

Construct the output Y by $Y = f(U)$. What is the distribution of Y?



Constructing a random variable by pushforwarding a measure - continuous

Answer: For a differentiable and strictly monotone function f, the density transformation formula is

$$p_Y(y) = \frac{p_U(f^{-1}(y))}{|f'(f^{-1}(y))|}.$$

Constructing a random variable by pushforwarding a measure - continuous

Answer: For a differentiable and strictly monotone function f, the density transformation formula is

$$p_Y(y) = \frac{p_U(f^{-1}(y))}{|f'(f^{-1}(y))|}.$$

Since U is uniform on $[0, 1)$, we have $p_U(u) = 1$ for $u \in [0, 1)$.

Applying the formula to each interval, where the slopes of f are $\frac{1}{2}$, 3, and $\frac{1}{2}$, respectively, we obtain

$$p_Y(y) = \begin{cases} 2 & \text{if } y \in [0, \frac{1}{8}), \\ \frac{1}{3} & \text{if } y \in [\frac{1}{8}, \frac{13}{8}), \\ 2 & \text{if } y \in [\frac{13}{8}, \frac{7}{4}), \\ 0 & \text{otherwise.} \end{cases}$$

Constructing a random variable by pushforwarding a measure - discrete

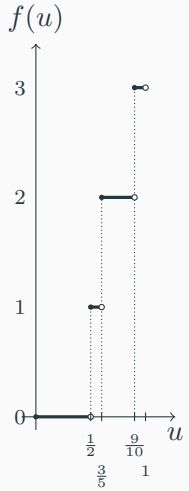
Example (Pushforward measure yielding a discrete distribution)

Suppose that U is a uniform random variable taking a value on $[0, 1)$.

Define $f : [0, 1) \to \{0, 1, 2, 3\}$ by the following:

$$f(u) = \begin{cases} 0 & \text{if } u \in [0, \frac{1}{2}), \\ 1 & \text{if } u \in [\frac{1}{2}, \frac{3}{5}), \\ 2 & \text{if } u \in [\frac{3}{5}, \frac{9}{10}), \\ 3 & \text{if } u \in [\frac{9}{10}, 1). \end{cases}$$

Construct the output Y by $Y = f(U)$. What is the distribution of Y?



Constructing a random variable by pushforwarding a measure - discrete

Answer:

Since $U \sim \text{Uniform}([0,1))$,

$$\begin{aligned}
\Pr(Y=0) &= \Pr\big(U \in [0, \tfrac{1}{2})\big) = 0.5, \\
\Pr(Y=1) &= \Pr\big(U \in [\tfrac{1}{2}, \tfrac{3}{5})\big) = 0.1, \\
\Pr(Y=2) &= \Pr\big(U \in [\tfrac{3}{5}, \tfrac{9}{10})\big) = 0.3, \\
\Pr(Y=3) &= \Pr\big(U \in [\tfrac{9}{10}, 1)\big) = 0.1.
\end{aligned}$$

Hence, Y follows the distribution defined by the PMF $P(0) = 0.5$, $P(1) = 0.1$, $P(2) = 0.3$, and $P(3) = 0.1$.

Property of a pushforward measure

We can observe the following:

Property of a pushforward measure

We can observe the following:

- Even one function can construct a random variable whose range is infinite set. This is hard by a PMF approach.

Property of a pushforward measure

We can observe the following:

- Even one function can construct a random variable whose range is infinite set. This is hard by a PMF approach.
- If the pushforwarding function f is Lipschitz continuous, it is hard to construct a discrete measure by pushforwarding. The pushforward approach is not suitable to construct a discrete measure.

Property of a pushforward measure

We can observe the following:

- Even one function can construct a random variable whose range is infinite set. This is hard by a PMF approach.
- If the pushforwarding function f is Lipschitz continuous, it is hard to construct a discrete measure by pushforwarding. The pushforward approach is not suitable to construct a discrete measure.

This is why a pushforward-measure approach is suitable for visual data generation.

Property of a pushforward measure

We can observe the following:

- Even one function can construct a random variable whose range is infinite set. This is hard by a PMF approach.
- If the pushforwarding function f is Lipschitz continuous, it is hard to construct a discrete measure by pushforwarding. The pushforward approach is not suitable to construct a discrete measure.

This is why a pushforward-measure approach is suitable for visual data generation.

- The output dimensionality of the function can be much smaller than the number of possible values.

Property of a pushforward measure

We can observe the following:

- Even one function can construct a random variable whose range is infinite set. This is hard by a PMF approach.
- If the pushforwarding function f is Lipschitz continuous, it is hard to construct a discrete measure by pushforwarding. The pushforward approach is not suitable to construct a discrete measure.

This is why a pushforward-measure approach is suitable for visual data generation.

- The output dimensionality of the function can be much smaller than the number of possible values.
- In visual data generation, a smooth distribution is acceptable. The distribution does not have to be discrete.

For latent diffusion approach

The motivation:

- Constructing a pushforward function that can be controlled by the input.
- Generate a low-dimensional tensor instead of raw high-dimensional output to minimize the computational cost.

The latent generation approach (e.g., latent diffusion model [4]) can achieve the above.

Latent generation pipeline

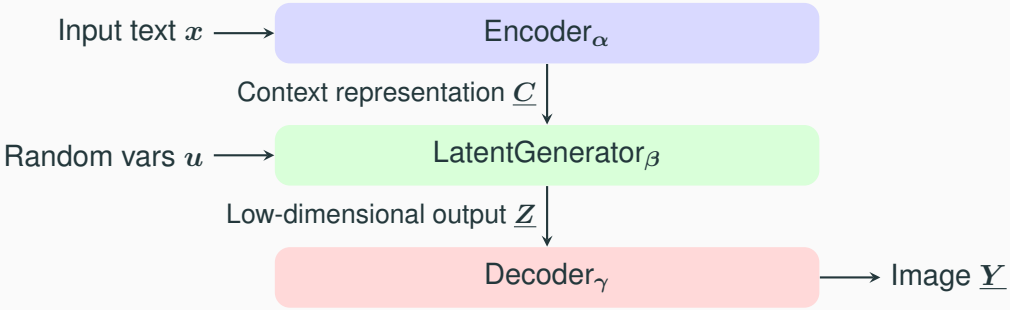


Figure 1: Latent generation pipeline.

Latent generation pipelines are used in many applications

- **Text-to-image**:
  - Stable diffusion v1.5
  - Qwen-Image 2.1
- **3D rendering**:
  - Hunyuan3D-2

Stable Diffusion 1.5 as a representative

As a representative example, we overview **Stable Diffusion v1.5**.

- Academically, one implementation of **Latent Diffusion Models** [4]
- One of the most practically derived image generation model[1].

- Encoder: **CLIP** text encoder (Transformer-based) [3].
- LatentGenerator$_\theta$: Generates the latent $\mathcal{Z}$ by **reverse diffusion**, iteratively applying **noise prediction** by a **U-Net** [5] [1, 4], iteratively receiving the context representation via **cross-attention**.
- Decoder: Reconstructs from latent to pixel space with a **VAE decoder** [2, 4].

[1] https://g-pc.info/archives/46460/

Alignment of this configuration with task characteristics

This three-part structure directly addresses the task characteristics we identified earlier.

Alignment of this configuration with task characteristics

This three-part structure directly addresses the task characteristics we identified earlier.

- **Batch Update of Enormous Output**: Computation is efficient because it **updates high dimensions all at once** without sequential tokenization (applying U-Net once per step).

Alignment of this configuration with task characteristics

This three-part structure directly addresses the task characteristics we identified earlier.

- **Batch Update of Enormous Output**: Computation is efficient because it **updates high dimensions all at once** without sequential tokenization (applying U-Net once per step).
- **Handling Under-determination**: **Diversity control** via $\mathcal{U}$ is built-in, and diverse solutions from the same condition can be generated naturally.

Alignment of this configuration with task characteristics

- **Reduction of Computational Cost**: Instead of outputting a high-dimensional image, the process outputs a low-dimensional latent variable and then converts it to a high-dimensional image with Decoder, thereby **reducing computational cost**.

Alignment of this configuration with task characteristics

Remark (Difference between Research Paper and General Implementation in Stable Diffusion 1.5)

Note the **difference between the research paper and general implementations**. The original paper on **Latent Diffusion Models** [4] explicitly states that the domain-specific encoder τ_{θ} (e.g., text encoder) for conditioning is **trained jointly** with the reverse diffusion network ϵ_{θ}.

Alignment of this configuration with task characteristics

Remark (Difference between Research Paper and General Implementation in Stable Diffusion 1.5)

Note the **difference between the research paper and general implementations**. The original paper on **Latent Diffusion Models** [4] explicitly states that the domain-specific encoder $\tau_{\boldsymbol{\theta}}$ (e.g., text encoder) for conditioning is **trained jointly** with the reverse diffusion network $\epsilon_{\boldsymbol{\theta}}$.

In fact, the paper states, **"both τ_θ and ϵ_θ are jointly optimized via Eq. 3."** and **"implement τ_θ as a transformer ... to infer a latent code"** (Sec. 3.3 and Sec. 4.3.1).

Alignment of this configuration with task characteristics

Remark (Continued)

On the other hand, the **Stable Diffusion** implementation in **Hugging Face Diffusers**, which is widely used in practice, states, **"This model uses a frozen CLIP ViT-L/14 text encoder to condition the model on text prompts."**[2], and uses the text encoder as **frozen**.

[2] https://huggingface.co/docs/diffusers/api/pipelines/stable_diffusion/overview

Alignment of this configuration with task characteristics

Remark (Continued)

On the other hand, the **Stable Diffusion** implementation in **Hugging Face Diffusers**, which is widely used in practice, states, **"This model uses a frozen CLIP ViT-L/14 text encoder to condition the model on text prompts."**[2], and uses the text encoder as **frozen**.

That is, there is an **operational difference** between the design principle of the original paper (whether it is jointly trained) and the representative implementation (using a fixed, existing encoder).

[2] `https://huggingface.co/docs/diffusers/api/pipelines/stable_diffusion/overview`

Practical visual data generation pipeline.

Question: How does a practical visual data generation pipeline avoid the high-dimensional output issues?

Answer: It adopt a pushforward approach. The **Generator** with the context representation converted by the **Encoder** pushforwards the base measure. The output will be converted to the raw data space by **Decoder**.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. In which sense are visual data generation different from Chat AI?

4. Why do we avoid constructing a probability mass function?

5. How does a practical visual data generation pipeline avoid the issues?

6. Summary and Future Plans

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. In which sense are visual data generation different from Chat AI?

4. Why do we avoid constructing a probability mass function?

5. How does a practical visual data generation pipeline avoid the issues?

6. Summary and Future Plans

Summary

Summary

- Visual data generation has **enormous output dimensions**, **weak local constraints**, and is **under-determined**.

Summary

- Visual data generation has **enormous output dimensions**, **weak local constraints**, and is **under-determined**.
- A pushforward measure is preferred if the data space has a continuous nature, where the number of possible values is extremely large and a continuous distribution is allowed.

Summary

- Visual data generation has **enormous output dimensions**, **weak local constraints**, and is **under-determined**.
- A pushforward measure is preferred if the data space has a continuous nature, where the number of possible values is extremely large and a continuous distribution is allowed.
- A diffusion pipeline is a pushforward approach in the low-dimensional latent space. It consists of a Encoder, a LatentGenerator$_\theta$, and a Decoder.

Next lecture

In subsequent lectures, we will detail each component.

Next lecture

In subsequent lectures, we will detail each component.

First, we will explain the **VAE**, which connects the raw high-dimensional image space and the latent low-dimensional space.

References i

[1] Jonathan Ho, Ajay Jain, and Pieter Abbeel.
Denoising diffusion probabilistic models.
In Advances in Neural Information Processing Systems (NeurIPS), 2020.

[2] Diederik P. Kingma and Max Welling.
Auto-encoding variational bayes.
arXiv preprint arXiv:1312.6114, 2013.

References ii

[3] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamila Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever.
Learning transferable visual models from natural language supervision.
In Proceedings of the 38th International Conference on Machine Learning (ICML), 2021.

[4] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer.
High-resolution image synthesis with latent diffusion models.
In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 10684–10695, 2022.

References iii

[5] Olaf Ronneberger, Philipp Fischer, and Thomas Brox.
U-net: Convolutional networks for biomedical image segmentation.
In Medical Image Computing and Computer-Assisted Intervention (MICCAI), pages 234–241, 2015.